

SOFTWARE ENGINEERING FOR MACHINE LEARNING · COMPANION READER

EDA, Model Registry & Serving Patterns

Architectural patterns that keep ML systems reliable and scalable

Session 6 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas; this reader gives you the *why*. Every slide concept is expanded into a full, plain-English explanation. Every slide example is worked out step by step. Colour-coded boxes guide you through each concept. The **blue** “intuition” box states the core idea in one breath. The **amber** “everyday picture” box ties it to real life before any math. The **green** “worked example” box solves a problem with real numbers and no skipped steps. The **red** “watch out” box flags the common mistake. The **purple** “key takeaway” box is the one sentence to remember. Read in order alongside the slides.

1 What this session covers

Session 6 asks one practical question: *how do we build ML systems that are reliable, scalable, and easy to maintain?* The answer comes in three architectural patterns.

1. **Event-Driven Architecture (EDA)** — how services talk to each other without being tightly joined. Components publish events; other components listen and react.
2. **Model Registry Pattern** — a central store for every version of every ML model. It bridges experimentation and production.
3. **Batch vs. Real-Time Serving** — two ways to deliver predictions. Batch runs offline on a schedule; real-time answers in milliseconds.

Each pattern solves a recurring problem. By the end you should be able to say *why* you would pick one over the other in a real project.

2 Event-Driven Architecture: services that talk without being glued

Picture a big software system with many moving parts: a web server, a recommendation engine, a notification service, a logging pipeline. If every part calls every other part directly, the system becomes a tangled web. Change one part and everything else may break. **Event-Driven Architecture (EDA)** is the pattern that cuts those direct connections. Instead of calling each other directly, services produce and consume **events** — small messages that say “something just happened.”

 The intuition

In EDA, services do not talk to each other directly. They shout into a shared room. Any service that cares can listen. Services that do not care can ignore the shout entirely. The room is the **event bus** or **message broker**.

★ Everyday picture

Think of a newspaper publisher. The publisher drops copies at a distribution centre. Subscribers pick up only the sections they care about: sport, business, or technology. The publisher has no idea who is reading. Readers have no idea who else is reading. They are *decoupled*.

In EDA, the publisher is a **producer**, the distribution centre is the **event bus**, and the readers are **consumers** (subscribers).

2.1 The two roles: producers and consumers

A **producer** (also called a *publisher*) is any service that generates an event. It might be a user clicking a button, an external API sending data, or an internal timer firing at midnight. The producer sends the event to the event bus and then forgets about it.

A **consumer** (also called a *subscriber*) is any service that registered its interest in a type of event. When the bus receives a matching event, it notifies the consumer. Many consumers can listen to the same event.

2.2 The event bus and message brokers

The **event bus** is the middleman. Real-world tools that play this role:

- **Apache Kafka** — high-throughput, fault-tolerant, built for streaming data at scale.
- **RabbitMQ** — flexible routing, good for task queues.
- **Amazon SQS / Azure Service Bus** — managed cloud services; no infrastructure to run yourself.

These tools store events even when a consumer is temporarily offline. When the consumer comes back, it catches up from where it left off.

2.3 Why EDA matters: the four problems it solves

The lecture lists four reasons to use EDA.

1. **Async communication.** Events happen at different times. EDA lets services process events when they are ready, not in lockstep.
2. **Decoupling.** Producers and consumers are independent. You can add a new consumer without touching the producer at all.
3. **Horizontal scalability.** Need to handle more events? Add more consumer instances. The bus handles the distribution.
4. **Reliability.** The broker stores events persistently. A consumer crash does not lose data; it resumes later.

Worked example: an e-commerce ML pipeline using EDA

An online store uses EDA to feed its recommendation model.

- Step 1. User action triggers an event.** A user clicks “Add to cart.” The web server (producer) publishes one event to the bus: `{user: 42, product: "headphones", event: "add_to_cart"}`.
- Step 2. The bus stores and routes the event.** Kafka writes the event to a topic called `user-interactions`. Any subscriber to that topic gets a copy.
- Step 3. Three consumers react in parallel.** Consumer 1 (ML pipeline) updates the user’s feature vector. Consumer 2 (analytics) increments a counter for headphones sold today. Consumer 3 (inventory) checks stock levels. None of the three knows about the others.
- Step 4. No consumer changes the producer.** The web server code never changed. Three consumers were added without touching the producer. This is the decoupling payoff.
- Step 5. Reliability check.** If Consumer 1 crashes during step 3, Kafka keeps the event. When Consumer 1 restarts, it reads from its saved offset and processes the event. No data is lost.

The ML pipeline, analytics, and inventory all stay in sync from one event, with no direct calls between any of them.

Event-Driven Architecture: producers publish events; the broker routes them to subscribers



Figure 1: The publish-subscribe flow in EDA. Producers push events to the broker. The broker routes each event to all registered consumers. No producer knows which consumers exist — that is the decoupling.

Watch out

- **Eventual consistency.** Consumers react async, so they may lag behind the producer. Do not assume all consumers have the latest state at any given instant.
- **Ordering is not free.** Events may arrive out of order. Kafka guarantees order within a partition, but you must set up partition keys carefully.
- **Debugging is harder.** A bug in a consumer is invisible to the producer. You need distributed tracing to follow an event’s path.

- **Overhead for simple systems.** Two services talking? A direct API call is simpler. EDA adds complexity; use it only when you need it.

Where this shows up in ML. An ML system is a *pipeline*: data collection, feature engineering, training, serving, monitoring. EDA lets each stage be an independent service. A new batch of training data triggers a `data-available` event. The feature store consumes it and fires `features-ready`. The training job starts. When done, a `model-trained` event fires. The serving layer picks it up and swaps the model. Each stage is replaceable. Tools like **Apache Kafka** and **MLflow** are often combined in exactly this pattern.

✓ Key takeaway

EDA decouples services through an event bus. Producers publish; consumers subscribe. No direct connections means each part can change, scale, and fail independently — the key property for a reliable ML system.

3 Design Patterns for ML Systems: why patterns exist

A **design pattern** is a named, reusable solution to a problem that comes up again and again. Patterns are not code you copy; they are templates you adapt. In ML engineering, patterns solve recurring challenges in three areas: data preparation, model training, and production serving.

👉 The intuition

A design pattern is a recipe. A recipe does not hand you a finished cake. It gives you a reliable sequence of steps for a class of cakes. You adapt it to your oven and your ingredients. ML design patterns do the same: they give you a proven structure for a recurring engineering problem.

Two patterns appear in almost every production ML project. They are the **Model Registry Pattern** and the **Batch vs. Real-Time Serving Pattern**.

4 The Registry Pattern: a controlled lookup table

Before the ML-specific model registry, it helps to see the general **registry pattern** from software engineering.

A registry is a “central hub to dynamically register, locate, and retrieve class instances or services.” Think of a phone book. You do not need to know someone’s address; you look up their name. The book is the registry.

★ Everyday picture

Imagine a hotel concierge desk. Every guest service (restaurant, taxi, spa) is listed there. If a guest wants a taxi, they ask the concierge, not the street. If a new restaurant opens, it registers with the concierge. The concierge is the registry: a single, up-to-date directory that both parties trust. Services find each other through the centre, not directly.

In ML, the registry pattern applies to *machine learning artifacts*: trained models, datasets, and experiment runs. The **ML Model Registry** is “a centralised repository that tracks, versions, manages, and governs ML models throughout their lifecycle.”

✓ Key takeaway

The registry pattern provides one authoritative place to register something once. Others can find it by name, version, or tag — without knowing where it physically lives.

5 Version Control: from source code to ML artifacts

You already know that **Git** tracks changes to source code. The model registry does the same job, but for ML artifacts. Understanding the gap between them explains why a separate tool is needed.

🔗 The intuition

Git stores files: code, config, and documentation. An ML registry stores *trained models*: large binary files. It also stores the metadata that makes them reproducible — hyperparameters, evaluation metrics, the dataset used, and the run ID. Storing a 2 GB weights file in Git is slow and wasteful. A registry is built for that job.

5.1 Three generations of version control

Version control tools evolved in three generations. The table below shows the key jump at each step.

Generation	Network	Operations	Example tools
First (software)	None	One file at a time	RCS, SCCS
Second (software)	Centralised	Multi-file	CVS, Subversion
Third (software)	Distributed	Changesets	Git
Third (ML)	Distributed + artifact tracking	Models, data, experiments	MLflow, DVC, W&B

The ML third generation adds *artifact tracking* on top of Git's distributed model. It stores code *and* the large binary outputs of training (model weights). It also stores metadata linking each output to its inputs: data version, hyperparameters, and metrics.

Three generations of version control — the fourth adds ML artifact tracking

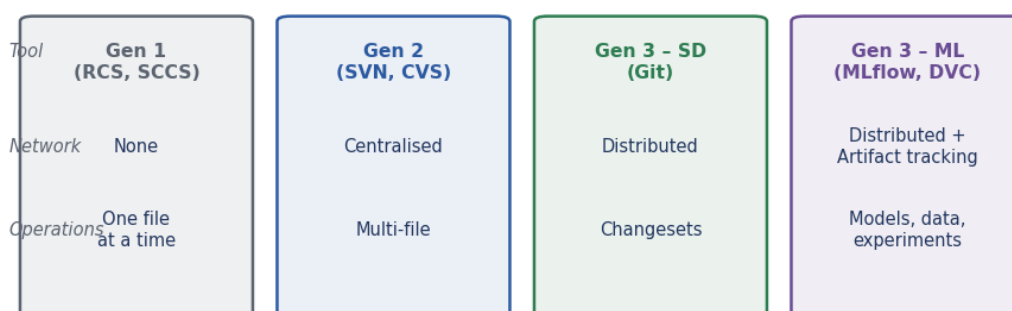


Figure 2: The four generations of version control, one per column. The ML generation adds artifact tracking on top of Git's distributed approach — that is the new capability.

 Worked example: what Git stores vs. what a model registry stores

A team builds a sentiment classifier. They use both Git and MLflow.

- Step 1. What goes into Git.** The training script `train.py`, the requirements file `requirements.txt`, and `config.yaml` go into Git. These are text files. The team can see every code change over time.
- Step 2. What goes into the ML registry.** After training run 42, MLflow stores several things. It stores the trained model weights (450 MB). It stores the hyperparameters: `lr=0.001`, `epochs=10`, `batch=32`. It stores the validation accuracy (0.912) and the Git commit hash of `train.py` so the run is reproducible.
- Step 3. The link between the two systems.** MLflow records the Git commit hash alongside the model weights. If someone asks “which code produced this model?” they look up the hash and check out that commit in Git. The two systems are linked, not merged.
- Step 4. Why not just Git?** Storing a 450 MB weights file in Git makes every clone slow and every push expensive. Git also does not know what “validation accuracy” means; MLflow does.

The two tools are complementary: Git for code, registry for artifacts.

6 Model Registry Pattern in MLOps

In MLOps, the model registry is “the definitive source of truth bridging the gap between model experimentation and production serving.” The registry is the *handover point*. It sits between the data scientist (who experiments) and the engineer (who deploys). That hand-off is its most important job.

 The intuition

Without a registry, deploying a model looks messy. A data scientist emails a weights file to an engineer. The engineer copies it to a server, hoping it is the right version. With a registry, the data scientist registers the model. The engineer promotes it to the *production* alias. The serving layer picks it up automatically. The whole process is auditable.

6.1 What a model registry stores

A model registry stores a set of linked artifacts for each model version.

- **Trained model weights** — the binary file(s) from training.
- **Datasets or dataset references** — which data was used, or a pointer to it.
- **Hyperparameters** — every setting used during training.
- **Metrics** — accuracy, F1, loss, or any evaluation number.
- **Experiment logs** — the training run that produced the model.
- **Lineage information** — which code, data, and run produced this exact artifact. This is what makes a model *reproducible*.

6.2 MLflow Model Registry: the four key features

MLflow is the most widely used open-source tool for ML experiment tracking and model registry. Its registry has four main features.

1. **Version control.** Every registration creates a new version number. You can compare versions, roll back, or run two versions in parallel for A/B testing.
2. **Model lineage and traceability.** Each version links back to the MLflow run that produced it. From the run you can find the code, data, and hyperparameters.
3. **Production-ready workflows.** MLflow supports *aliases* (e.g. `production`, `staging`, `champion`) and *tags* (e.g. `approved=true`). To promote a model, set its alias. The serving layer reads the alias, not a version number. It picks up the new model automatically.
4. **Governance and compliance.** The registry keeps a full audit log of every promotion, annotation, and access event. This matters in regulated industries where you must prove which model made which decision at what time.

Model Registry: the lifecycle from experiment to production and retirement



Figure 3: The model lifecycle in an ML registry. A model moves left to right as it gains confidence. The registry tracks every transition with a timestamp and a user ID.

Worked example: promoting a model from staging to production in MLflow

A team has trained a new churn-prediction model (version 3). They want to replace the current production model (version 2).

- Step 1. Register the new model.** After training run 107, the data scientist calls: `mlflow.register_model(run_uri, "churn-model")`. MLflow creates `churn-model` version 3 with status *None* (not yet promoted).
- Step 2. Validate in staging.** The team sets the alias: `client.set_registered_model_alias("churn-model", "staging", 3)`. The staging server reads the `staging` alias and loads version 3. QA tests pass: $F1 = 0.87$ vs. version 2's 0.82.
- Step 3. Promote to production.** With approval, the team updates the alias: `client.set_registered_model_alias("churn-model", "production", 3)`. The production endpoint reads the `production` alias. It now serves version 3. **No code change** was needed.
- Step 4. Keep version 2 as a fallback.** Version 2 stays in the registry with no alias.

If version 3 behaves badly, the team reassigns **production** back to version 2 in seconds.

Step 5. Audit trail. Every alias change is logged: who did it, when, and the previous value. This satisfies compliance requirements.

The whole promotion took five steps and zero code deployments.

⚠ Watch out

- **The registry is not a serving layer.** It stores models; it does not run them. You still need a separate serving infrastructure that *reads from* the registry.
- **Registry metrics are training metrics.** They do not tell you how the model behaves in production. Add a monitoring layer that tracks live performance separately.
- **Version numbers can mislead.** Version 5 is not always better than version 4. Always compare metrics; never assume a higher version number is better.
- **Lineage is only as good as your logging.** If a training run did not log its dataset version, the registry has an incomplete lineage record. Build logging into the training code from the start.

Where this shows up in ML. Every serious MLOps platform uses a model registry at its core. **MLflow Registry**, **Weights & Biases Model Registry**, **DVC**, and cloud-native registries (**AWS SageMaker**, **GCP Vertex AI**, **Azure ML Registry**) all implement this pattern. When you read about “CI/CD for ML” or “model promotion workflows,” the model registry is the component that makes those workflows possible. Without it, there is no reliable way to know which model is in production or to roll back when something goes wrong.

✓ Key takeaway

The model registry is the single source of truth for ML models in production. It tracks every version, every lineage link, and every promotion decision. The serving layer reads from the registry by alias — so promoting a model requires no code change, just a registry update.

7 Batch vs. Real-Time Serving: two ways to deliver predictions

Once you have a model in the registry, you need to serve it: run it on new data and return predictions. There are two fundamental patterns. The right choice depends on how quickly the business needs the answer.

🧠 The intuition

Batch serving is like a bakery: bake a hundred loaves overnight, stock the shelves in the morning, sell them through the day. Real-time serving is like a food truck: each customer orders, you cook it fresh, and you hand it over in two minutes. Both are valid. The question is what your customer needs.

7.1 Batch prediction: processing data in bulk

Batch prediction (also called *offline prediction*) runs the model on a large set of observations at once. It is a scheduled job, not a live service. The results are stored in a database or file and consumed later.

The lecture gives a clear example: send a daily promotional email to customers likely to buy a product. You do not need that list right now. A nightly job can score all customers, save the top 1,000, and let the marketing team use the list next morning.

When to use batch prediction:

- **Customer segmentation** — group customers by behaviour over the past week. No one needs the segments in milliseconds.
- **Product recommendations in emails** — generate personalised recommendations for a daily newsletter. The email goes out once a day.
- **Risk reporting** — compute credit risk scores for all loan applicants at end of business. The decision is made the next morning.

★ Everyday picture

A payroll system runs once a week. Every salary is computed in one batch job on Friday night. No one needs their exact pay-to-date figure at 3 pm on Wednesday. The batch job is fast, cheap, and perfectly correct for this use.

7.2 Real-time (online) prediction: serving on demand

Online prediction (also called *real-time* or *on-demand inference*) deploys the model as a persistent web API. The service is always running. When a request arrives, the model scores it and returns a prediction in milliseconds.

The lecture's example is a credit card fraud check. The merchant needs “approve” or “deny” *before the customer walks away*. A batch job running tonight is useless. The model must be a live API that answers every transaction in under 200 ms.

When to use real-time prediction:

- **Fraud detection** — score a transaction before it clears.
- **Dynamic pricing** — adjust ride-share prices based on current supply and demand, updated every few seconds.
- **Spam filtering** — decide if an incoming email is spam *before* it reaches the user's inbox.

Worked example: choosing the right serving pattern for three ML use cases

For each use case, decide: batch or real-time?

Step 1. Weekly newsletter recommendations. The email goes out every Monday at 9 am. There is all weekend to score customers. Millions of customers need scoring. **Verdict: batch.** Run a Spark job on Sunday night. Store the top-5 recommendations per customer in a database. The email service reads the database on Monday. No live API needed.

Step 2. Autocomplete for a search bar. The user types one character. Suggestions must appear in under 50 ms. A new request comes with every keypress. **Verdict: real-time.** Deploy the model as a REST API (e.g. FastAPI). Each keypress sends an HTTP request; the API replies in under 50 ms. The model stays loaded in memory at all times.

Step 3. Monthly churn report for account managers. Each month, account

managers get a list of clients at high risk. They review it in a monthly meeting.
Verdict: batch. Run a scoring job on the last day of every month. Export results to a spreadsheet. The manager opens it next morning. A real-time API would be wasted here.

Step 4. Match the pattern to the need. If you picked real-time for all three, you would waste engineering effort maintaining live APIs for two use cases that never need them. If you picked batch for the search bar, the product would not work. Always ask: *how fresh must the prediction be?*

7.3 Comparing the two patterns side by side

Property	Batch	Real-time
When predictions run	Scheduled (hourly, daily, weekly)	On each incoming request
Latency	Minutes to hours; result consumed later	Milliseconds
Infrastructure	Spark, Airflow, a cron job	A persistent API server (FastAPI, TensorFlow Serving)
Cost profile	Lower — runs when compute is cheap	Higher — server always on
Data freshness	Stale by design (last batch window)	Fresh at request time
Typical use cases	Emails, reports, segmentation	Fraud, search, real-time recommendations

Batch vs real-time: trade freshness for complexity and cost

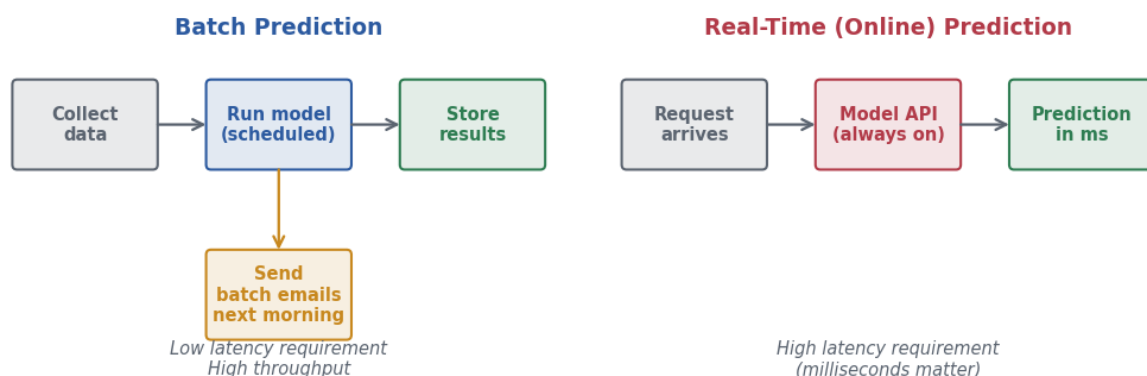


Figure 4: Batch vs. real-time serving side by side. Left: a scheduled job stores results for later use. Right: a live API answers every request in milliseconds. Neither is always better — pick by latency need.

X Watch out

- **Batch latency is a hard constraint.** If the batch job runs at midnight, predictions are up to 24 hours old. If the business needs fresher data, shorten the batch window or

switch to real-time.

- **Real-time has cold-start costs.** A server that is always on costs money even when idle. For low-traffic cases, a serverless function (AWS Lambda) can serve predictions without a 24/7 server. The trade-off: the first request after an idle period is slow (that delay is called the *cold start*).
- **Mixing patterns is common.** Many systems use batch to pre-compute features and real-time to score them. The batch job runs nightly to build feature vectors; the live API reads those vectors from a feature store at request time. This is called a *lambda architecture*.
- **Real-time is harder to debug.** A batch job writes a log file you can inspect. A live API at 1,000 requests/second produces a flood of logs. Build observability (metrics, tracing, alerting) from day one.

Where this shows up in ML. This is one of the most-asked questions in ML system design interviews: “Would you use batch or real-time serving for ...?” Almost every major ML platform exposes both patterns. **AWS SageMaker** has *Batch Transform* (batch) and *Real-Time Inference Endpoints*. **GCP Vertex AI** has *Batch Prediction* and *Online Prediction*. **TensorFlow Serving** and **Triton Inference Server** are common real-time engines. **Apache Spark** and **Airflow** are common batch engines. Knowing which to use, and when to combine them, is a core MLOps skill.

✓ Key takeaway

Batch prediction runs on a schedule and stores results for later. Real-time prediction keeps a live API that answers in milliseconds. Choose by asking: *how fresh must the prediction be when needed?* If hours are fine, use batch. If seconds matter, use real-time.

Glossary & symbols

Key terms.

event	a small message saying “something just happened.” Published by a producer; consumed by one or more subscribers.
event bus	the shared channel that receives and routes events. Real tools: Kafka, RabbitMQ, Amazon SQS.
producer	a service that generates and publishes events. Also called a <i>publisher</i> .
consumer	a service that subscribes to and processes events. Also called a <i>subscriber</i> .
decoupling	the property that services do not call each other directly. Each can change, scale, or fail independently.
design pattern	a named, reusable solution to a recurring engineering problem. A template, not code you copy.
model registry	a central store for ML model artifacts, versions, metadata, and lineage. It bridges experimentation and production.
model lineage	the full chain from raw data to deployed model: which data, code, and

	hyperparameters produced this artifact.
alias	a human-readable name (e.g. production) that points to a specific model version. Promotion means reassigning the alias.
batch prediction	running a model on a large dataset at once, on a schedule, storing results for later use.
online prediction	running a model via a live API that answers each request in milliseconds.
latency	the time between a request arriving and the response returning. The key metric for real-time serving.
cold start	the delay on the first request after an idle period (common in serverless real-time serving).
lambda architecture	a system that combines batch and real-time: batch for heavy offline work, real-time for fast online lookups.

Tools at a glance.

Tool	Role in this session
Apache Kafka	Event bus for high-throughput event-driven systems
RabbitMQ	Message broker; flexible routing and task queues
Amazon SQS	Managed cloud message queue
MLflow	Experiment tracking and model registry (open-source)
DVC	Data and model version control (Git-compatible)
Weights & Biases	Experiment tracking and model registry (cloud)
Apache Spark	Distributed batch processing engine
Apache Airflow	Workflow orchestrator for batch pipelines
FastAPI	Python web framework; common for real-time serving
